LUS Car Automation — Track 1

Three-Day Progress Report

Real-Car OBD-II Data Reading, Parts Purchase, and Planning for CAN Data

Days covered: Friday 10 July, Monday 13 July, and Tuesday 14 July 2026

---

1. Summary

Over these three working days, we moved our work from testing on the computer to reading data from a real car. On Friday, three of us went to a Maruti Suzuki Fronx and read live engine data from its OBD-II port using our Bluetooth ELM327 adapter, while the other three members went out to buy the parts we needed. On Monday, we bought the remaining main parts for our in-car device, and at the same time we finalised our 32-byte data packet. On Tuesday, we brought all the parts together, studied the adapter to plan the wiring, and decided how we will read the extra data in the next stage using the car's CAN bus along with the OBD-II port.

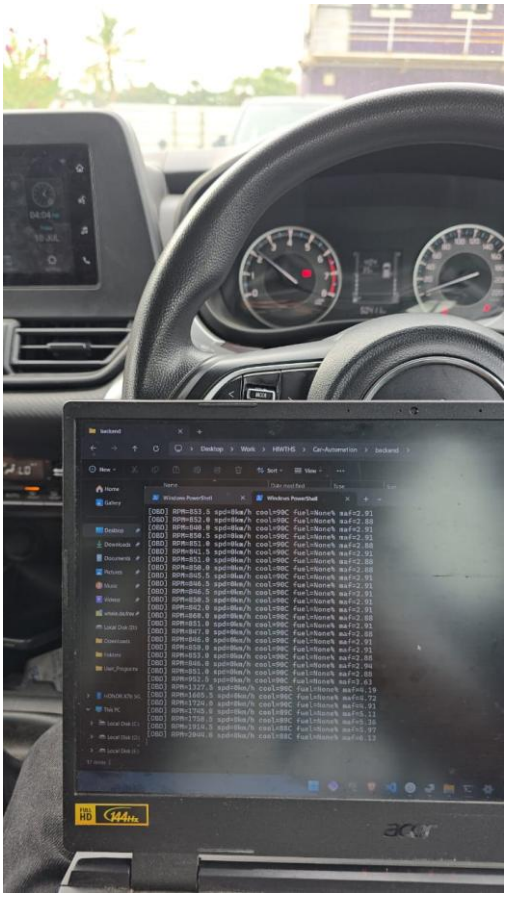
Main result: the car talks to our adapter and our decoding is correct. RPM, coolant temperature, engine load, throttle, MAF, and intake temperature all read correctly on the real car. Two things are still left to finish, and we have noted them clearly below. First, we have not yet checked vehicle speed properly, because all our testing so far was done while the car was parked. Second, a few fields in the packet are still showing placeholder values, which we already understand and plan to correct.

2. Friday 10 July — Testing on the Real Car

How we split the work: three of us went to the car with the laptop and the adapter, and the other three went to buy parts (the microSD card purchase is shown in Section 5).

2.1 What we did

We plugged our Bluetooth ELM327 Mini adapter into the Fronx's OBD-II port and paired it to the laptop through a Windows COM port. Our program set up the adapter, let it find the car's protocol automatically, and asked for our eight target PIDs once every second. We did this test with the car at idle in neutral, and we pressed the accelerator now and then to raise the RPM and check that the readings changed correctly.



Live capture inside the Fronx: engine running, ignition on, RPM streaming into the terminal. The instrument cluster and centre screen (clock 04:04, 10 Jul) confirm the on-vehicle session.

2.2 What worked well

The adapter connected without trouble and the car replied to every PID we asked for. The values we got made sense for a warm engine at idle, and they changed correctly when we pressed the accelerator:

- **RPM:** correct. It stayed around 840 to 860 at idle and went above 2,000 when we revved the engine, matching what we could hear.
- **Coolant:** 88 to 90 °C, which is normal for a warmed-up engine.
- **MAF (air flow):** went up and down with the RPM. This gave us a second, separate way to be sure our decoding was real and not a coincidence.
- **Engine load, throttle, and intake temperature:** all gave sensible values.

We also saved the raw hex replies from the adapter (for example, **41 0C 20 B2** gives 2,092 RPM). Saving the raw hex along with the decoded values means that if we ever have a doubt about our decoding later, we can check it again against the original data from the car.

2.3 What is still left to finish

Vehicle speed not yet checked. Since the car was parked in neutral, the speed showed 0 km/h the whole time. This is normal for a parked car, but it means speed is the one main reading we have not confirmed yet. The first thing we will do on our next drive is to check that the speed value changes along with the car's speedometer while moving.

Fuel level is not given by this car. Every reading returned a negative reply for fuel level. This is simply how the Fronx is built, and it does not share fuel level on the OBD-II port. In this case the correct thing to do is to mark it as not available.

3. Monday 13 July — Buying the Main Parts and Finalising the Packet

3.1 Buying the parts

We bought the remaining main parts for our in-car device in one order. This included the microcontroller, the CAN module, the GPS module, and the breadboards and soldering items we need to build the device. The full bill is shown in Section 5.

Main items: ESP32 board, MCP2515 CAN module, NEO-6M GPS module, two breadboards, two large PCBs, jumper wires, and soldering items (iron, paste, lead).

3.2 Finalising the 32-byte packet

At the same time, we finalised the design of our data packet. Our earlier version had a few fixed placeholder values (like a fixed battery voltage and a fixed fault-code count) and could only reach the data available on the normal OBD-II port. Our final design is built around the data we actually need, and it replaces the fixed values with real readings or with fields we have honestly kept empty for now.

The data we need, and how we get each one:

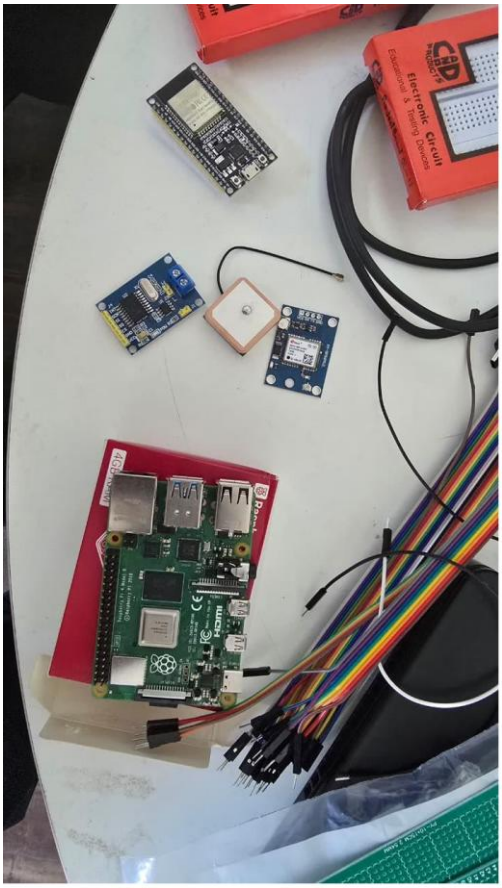
Data we need	How we get it	Present status
Speed	OBD-II (standard)	Working; to check while moving
RPM	OBD-II (standard)	Working, checked on the car
Fuel mileage	Calculated from MAF and speed	Calculated at the backend
Gear	Calculated from RPM and speed	Rough estimate for now
GPS position	NEO-6M module	Module ready; to add to packet
AC running	Car CAN bus	To be found in next stage
Brake pedal	Car CAN bus	To be found in next stage
Clutch (on/off)	Car CAN bus	To be found in next stage

A small note: *we changed the clutch requirement from a percentage to a simple on or off value, because a continuous clutch sensor is not expected on this car. We also left out driver-communication, since that is something we build as an output and not a value we read from the car.*

4. Tuesday 14 July — Bringing Parts Together and Planning the CAN Stage

4.1 Bringing the parts together

We brought all our parts together and laid them out: the Raspberry Pi 4 (4 GB), the ESP32 boards, the NEO-6M GPS with its antenna, the MCP2515 CAN module, breadboards, and our earlier build. We also confirmed that the microSD card and the Pi power supply are ready, so there is nothing stopping us from starting the Pi for the first time.



All the parts for our in-car device: Raspberry Pi 4, ESP32, NEO-6M GPS module, MCP2515 CAN module, and other supplies.

4.2 Studying the adapter for wiring

We opened the ELM327 adapter to plan how it and the CAN module can share one OBD connector. We found that because the CAN bus (pins 6 and 14) is a shared line by design, both the ELM327 and the MCP2515 can connect to the same two lines at the same time, both only listening, without any problem. Our plan is to bring out pins 6, 14, and ground from one OBD head and connect both devices from a common point.

4.3 Deciding the plan for the next stage

The main decision of the day was to use both the OBD-II port and the car's CAN bus together, instead of only the OBD-II port. The reason is simple: three of the values we need, which are AC, brake, and clutch, are not given on the normal OBD-II port. They are only present as messages on the car's CAN bus, so to read them we need to listen to that bus and find out which message carries which value.

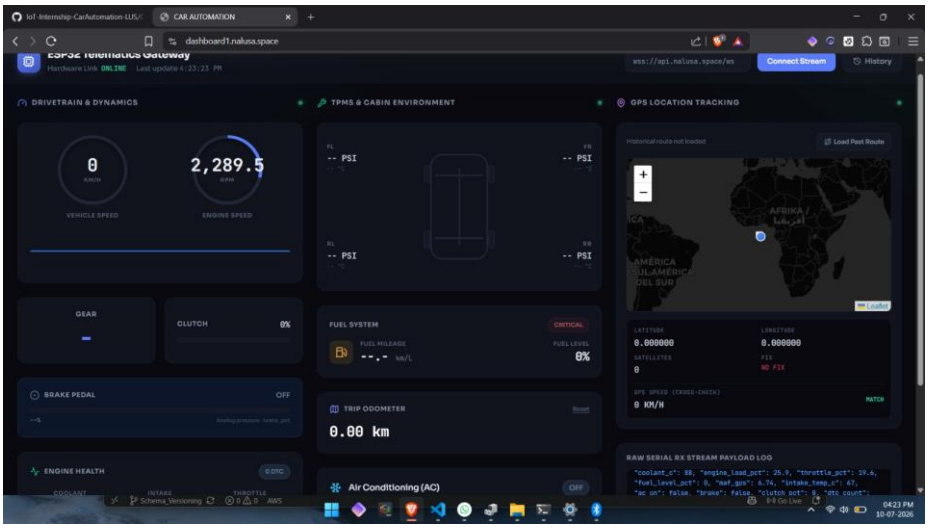
- **OBD-II side (working):** the ELM327 gives us the standard readings, such as RPM, speed, coolant, load, throttle, MAF, and intake temperature.

- **CAN side (new):** the MCP2515, read by the ESP32 in listen-only mode, records the raw messages. We save these and later compare them (for example, brake pressed against brake released) to find the AC, brake, and clutch values.
- **Putting them together:** both go to the Raspberry Pi, which stores them and joins them into the 32-byte packet once we have found the CAN values.

Safety point: our CAN reading is only listening. The device never sends anything to the car's bus; it only observes. We keep this as a firm rule for anything we connect to a running car.

4.4 Current dashboard (showing what is done and what is left)

We ran our live dashboard using a capture. It is helpful because it shows both the working data and the things still left to finish in one screen:



Our live dashboard. The RPM (2,289.5) is real and correct. The other marked points are the known placeholder items, not problems with the car.

- **Working:** engine RPM shows live and correct.
- **Speed 0 km/h:** this was a parked test, so speed is still to be checked while moving.
- **Fuel 0%:** this is a placeholder. The car does not give fuel level, and our packing step currently fills the empty value with zero, so the dashboard shows it as an empty tank. We plan to correct this by marking missing values as not available instead of zero.
- **GPS no fix at 0,0:** GPS is not yet added to this capture, so the position is a placeholder. The marker showing near Africa is just the 0,0 placeholder, and it reminds us that adding GPS is our next step.
- **Clutch, AC, and brake:** these show default values for now, because they come from the CAN side, which is the work for the next stage.

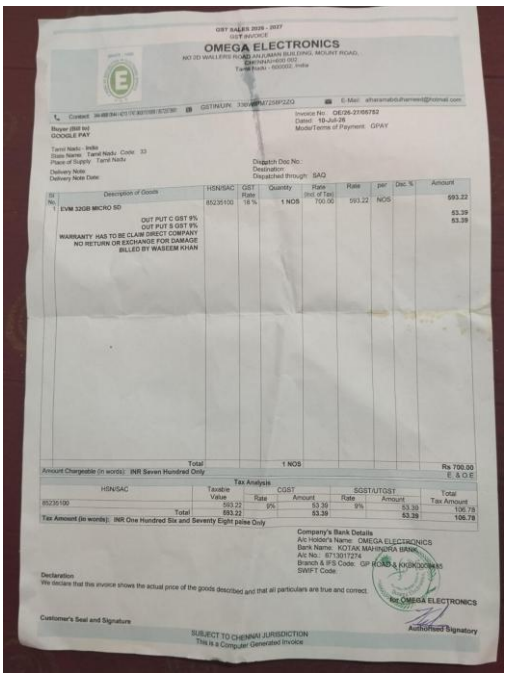
5. Record of Purchases

All the parts were billed to Let Us Support Private Limited. Two bills cover these days.

5.1 Friday 10 July — Omega Electronics

Item	Qty	Amount (₹)
32 GB microSD card (Pi boot/storage)	1	700.00

Invoice total: ₹700.00 (incl. GST). Invoice OE/26-27/05752, dated 10-Jul-26.

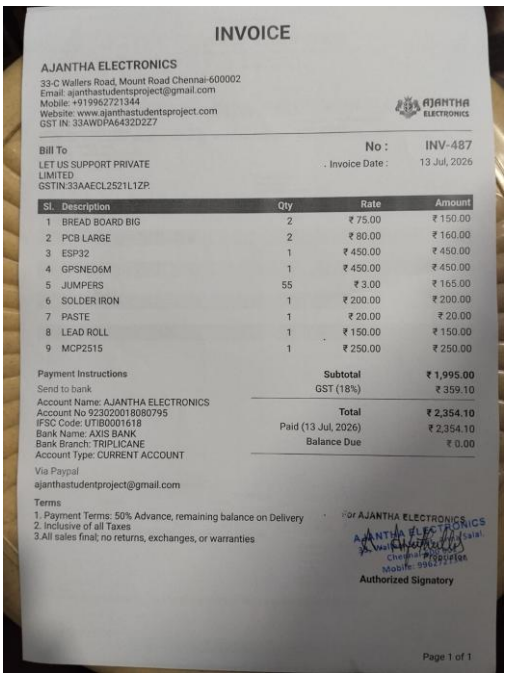

OMEGA ELECTRONICS

Omega Electronics invoice — microSD card.

5.2 Monday 13 July — Ajantha Electronics (INV-487)

Item	Qty	Rate (₹)	Amount (₹)
Breadboard (big)	2	75.00	150.00
PCB (large)	2	80.00	160.00
ESP32	1	450.00	450.00
GPS NEO-6M	1	450.00	450.00
Jumpers	55	3.00	165.00
Solder iron	1	200.00	200.00
Paste	1	20.00	20.00
Lead roll	1	150.00	150.00
MCP2515 (CAN)	1	250.00	250.00

Subtotal ₹1,995.00 + GST 18% ₹359.10 = Total ₹2,354.10 (paid). Invoice INV-487, dated 13-Jul-26.



INVOICE

AJANTHA ELECTRONICS
33-C Wallers Road, Mount Road Chennai-600002
Email: ajanthastudentsproject@gmail.com
Mobile: +919962721344
Website: www.ajanthastudentsproject.com
GST IN: 33AWDPA6432D2Z7

AJANTHA ELECTRONICS

Bill To
LET US SUPPORT PRIVATE LIMITED
GSTIN:33AAECL2521L1ZP.

No : INV-487
. Invoice Date : 13 Jul, 2026

Sl.	Description	Qty	Rate	Amount
1	BREAD BOARD BIG	2	₹ 75.00	₹ 150.00
2	PCB LARGE	2	₹ 80.00	₹ 160.00
3	ESP32	1	₹ 450.00	₹ 450.00
4	GPSNEO6M	1	₹ 450.00	₹ 450.00
5	JUMPERS	55	₹ 3.00	₹ 165.00
6	SOLDER IRON	1	₹ 200.00	₹ 200.00
7	PASTE	1	₹ 20.00	₹ 20.00
8	LEAD ROLL	1	₹ 150.00	₹ 150.00
9	MCP2515	1	₹ 250.00	₹ 250.00

Payment Instructions
Send to bank
Account Name: AJANTHA ELECTRONICS
Account No 923020018080795
IFSC Code: UTIB0001618
Bank Name: AXIS BANK
Bank Branch: TRIPLICANE
Account Type: CURRENT ACCOUNT

Via Paypal
ajanthastudentproject@gmail.com

Subtotal	₹ 1,995.00
GST (18%)	₹ 359.10
Total	₹ 2,354.10
Paid (13 Jul, 2026)	₹ 2,354.10
Balance Due	₹ 0.00

Terms
1. Payment Terms: 50% Advance, remaining balance on Delivery
2. Inclusive of all Taxes
3.All sales final; no returns, exchanges, or warranties

For AJANTHA ELECTRONICS
Authorized Signatory

Page 1 of 1

Ajantha Electronics invoice — core device hardware.

Combined spend for the period: ₹3,054.10.

6. Plan for Tomorrow and Next Steps

Tomorrow we plan to go to the site again and complete the full 32-byte data reading from the car. We also plan to make a start on car acceleration, and we will try it using two different methods to see which one suits us better. We will report the results of both once we have tested them.

Along with this, our next steps are:

- **Check the vehicle speed while the car is moving** — this is the one main reading we still have to confirm.
- **Read the car's list of supported values and its vehicle number** — this gives us a clear record of exactly what this car provides.
- **Correct the placeholder values** — show missing values (like fuel) as not available instead of zero, so the dashboard no longer shows wrong readings.
- **Add GPS to the packet** — replace the 0,0 placeholder with the real position from the NEO-6M module.
- **Do the CAN listening test** — connect the MCP2515 to the CAN lines and check that the raw messages appear. If they do, we can begin finding the AC, brake, and clutch values.